

JavaScript-14

Avancerad React JS

26 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

Avancerad React JS

Nu när du vet hur du installerar React och hur en enskild komponent fungerar, ska vi titta på hur man bygger appar på riktigt – genom att skicka data mellan komponenter och hämta information från ett externt REST API.

Props – Skicka data mellan Lego-bitar

I React vill vi ofta återanvända komponenter. Tänk dig att du vill visa tre olika produktkort på en sida. Istället för att skapa tre olika filer, skapar vi *en* komponent som heter ProduktKort, och sedan skickar vi med olika information till den.

Informationen vi skickar med kallas för **Props** (properties). Det fungerar precis som när du skickar med argument in i en vanlig JavaScript-funktion.

Här skapar vi först en liten underkomponent som tar emot props:

```
// En fristående komponent som tar emot ett objekt (props)
function ProduktKort(props) {
  return (
    <div className="kort">
      <h3>{props.titel}</h3>
      <p>Pris: {props.pris} kr</p>
    </div>
  );
}
```

Bild 1 En fristående komponent i React.

Inuti vår huvudfil (App.jsx) kan vi nu använda den här komponenten flera gånger och skicka med olika props till den:

```
function App() {
  return (
    <div className="app-container">
      <h1>Vår Butik</h1>

      {/* Vi skickar med data som attribut i JSX */}
      <ProduktKort titel="Blå Jeans" pris={799} />
      <ProduktKort titel="Röd T-shirt" pris={299} />
    </div>
  );
}
```

Bild 2 Props gör att samma komponent kan visa helt olika innehåll..

Hämta data från ett API (Fetch i React)

I dina tidigare kapitel lärde du dig hur man hämtar data med `fetch()`. I React kan vi inte bara kasta ut ett `fetch`-anrop hur som helst inuti en komponent, eftersom komponenten ritas om varje gång ett state ändras (vilket skulle starta en oändlig loop av API-anrop!).

För att styra *när* kod ska köras använder vi en inbyggd React-funktion som heter **useEffect**. Vi talar om för React: "Kör det här `fetch`-anropet en enda gång, precis när sidan laddas för första gången". Här hämtar vi en lista med användare från ett gratis REST-API (JSONPlaceholder).

```
import { useState, useEffect } from 'react';

function AnvändarLista() {
  const [användare, setAnvändare] = useState([]); // Startar med en tom array

  // useEffect tar två argument: en funktion och en "dependency-array" []
  useEffect(() => {

    // Vi gör vårt fetch-anrop inuti useEffect
    fetch('https://typicode.com')
      .then(response => response.json())
      .then(data => {
        setAnvändare(data); // Vi sparar datan i vårt state!
      });

  }, []); // Den tomma klammern [] i slutet betyder: "Kör bara EN gång vid sidstart"

  return (
    <div>
      <h2>Våra Kunder</h2>
      <ul>
        { /* Vi använder .map() från kapitel 06 för att loopa ut datan! */ }
        { användare.map(user => (
          <li key={user.id}>{user.name} ({user.email})</li>
        )) }
      </ul>
    </div>
  );
}
```

Bild 4: Kombinerat `useEffect` för att hämta data och `useState` för att spara den.
